Contents

1. Lecture recap - Suchen und Sortieren
   1.1. Binäre Suche
   1.2. Bubble-Sort
   1.3. Selection-Sort
   1.4. Insertion-Sort
   1.5. Mergesort
   1.6. Quicksort
   1.7. Max-Heap
2. Moodle Quiz Besprechung
3. Häufige Fehler aus Exercise Sheet 2.
4. In-Class Aufgaben

1. Lecture recap - Suchen und Sortieren

1.1. Binäre Suche

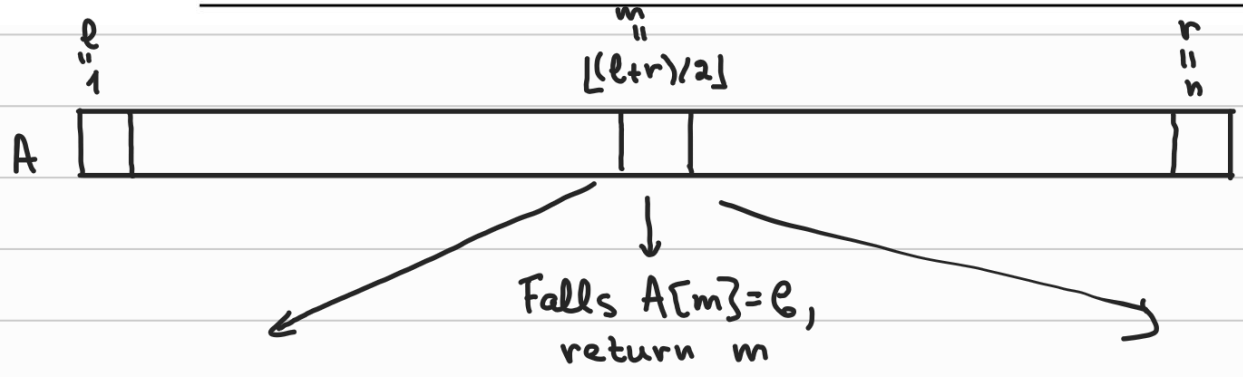
Annahme: $A[1...n]$ ist sortiert mit $A[1] \leq A[2] \leq A[3] \leq .... \leq A[n]$

Wir sind $A[1...n]$ und ein Schlüssel b gegeben und wir suchen nach ein Index k, so dass $A[k] = b$. Falls kein solchen Index existiert, geben wir "nicht gefunden" zurück.

Algorithmus:

BINÄRE SUCHE — BINARY-SEARCH$(A[1..n], b)$

```
1 ℓ ← 1; r ← n                          ▷ Initialer Suchbereich
2 while ℓ ≤ r do
3     m ← ⌊(ℓ + r)/2⌋
4     if A[m] = b then return m          ▷ Element gefunden
5     else if A[m] > b then r ← m − 1    ▷ Suche links weiter
6     else ℓ ← m + 1                     ▷ Suche rechts weiter
7 return "Nicht vorhanden"
```



Falls $A[m] > e$, dann suchen wir e in der linken Teil von A

$\Rightarrow$ ℓ bleibt unverändert

$r = m - 1$

Gefunden!

Falls $A[m] < e$, dann suchen wir e in der rechten Teil von A

$\Rightarrow$ $\ell = m + 1$

r bleibt unverändert

Laufzeit: $O(\log n)$

1.2. Bubble-Sort

BUBBLE-SORT($A[1..n]$) — BUBBLE-SORT

```
1 for j ← 1, 2, ..., n do
2     for i ← 1, 2, ..., n − 1 do
3         if A[i] > A[i + 1] then
4             tausche A[i] und A[i + 1]
```

Beispiel. *Betrachten wir das Array* $A = (5, 3, 7, 1, 4)$. *Bubble-Sort führt nun die folgenden Schritte durch:*

$j = 1$:	5	3	7	1	4	3	5	7	1	4	3	5	7	1	4	3	5	1	7	4	3	5	1	4	7
$j = 2$:	3	5	1	4	7	3	1	5	4	7	3	1	4	5	7	3	1	4	5	7	3	1	4	5	7
$j = 3$:	3	1	4	5	7	1	3	4	5	7	1	3	4	5	7	1	3	4	5	7	1	3	4	5	7

Wir beobachten, dass Bubble-Sort in diesem Fall bereits nach drei Durchläufen das Array sortiert hat, obwohl $n = 5$ *ist.*

Invariant: Nach j Durchläufen der äusseren Schleife sind die j grösste Elemente am richtigen Ort.

Laufzeit:

- $\Theta(n^2)$ Vergleiche
- $O(n^2)$ Vertauschungen

Insgesamt $O(n^2)$

1.3. Selection - Sort

Algorithm:

SELECTION-SORT($A[1..n]$) — SELECTION-SORT

```
1 for j ← n, n − 1, . . . , 1 do
2     k ← Index des Maximums in A[1, . . . , j]
3     tausche A[k] und A[j]
```

Wir beginnen mit j=n und auf jede Iteration müssen wir das grösste Element im Teilarray A[1,...,j] finden und dies mit A[j] vertauschen

Beispiel. *Betrachten wir das Array* $A = (5, 3, 7, 1, 4)$. *Selection-Sort führt nun die folgenden Schritte durch:*

$$
\begin{aligned}
j = 5: &\quad (5, 3, \textcolor{red}{7}, 1, 4) \to (5, 3, 4, 1, \textcolor{red}{7}) \\
j = 4: &\quad (\textcolor{red}{5}, 3, 4, 1, 7) \to (1, 3, 4, \textcolor{red}{5}, 7) \\
j = 3: &\quad (1, 3, \textcolor{red}{4}, 5, 7) \to (1, 3, \textcolor{red}{4}, 5, 7) \\
j = 2: &\quad (1, \textcolor{red}{3}, 4, 5, 7) \to (1, \textcolor{red}{3}, 4, 5, 7) \\
j = 1: &\quad (\textcolor{red}{1}, 3, 4, 5, 7) \to (\textcolor{red}{1}, 3, 4, 5, 7)
\end{aligned}
$$

Wir beobachten, dass Selection-Sort in diesem Fall bereits nach zwei Durchläufen das Array sortiert hat.

Invariante: Nach j Durchläufen der äusseren Schleife sind die j grösste Elemente am richtigen Ort.

Laufzeit:

- $\Theta(n^2)$ Vergleiche,
- $O(n)$ Vertauschungen

Insgesamt: $O(n)$

1.4. Insertion - Sort

Invariante: Nach j Durchläufen der äusseren Schleife ist das Teilarray A[1,...,j] sortiert (es enthält nicht zwangsläufig die j kleinsten Elemente des Arrays)

Formaler ausgedrückt gilt also:

$$A[1] \leq A[2] \leq \ldots \leq A[j]$$

Algorithmus:

INSERTION-SORT — INSERTION-SORT($A[1..n]$)

```
1 for j ← 2, 3, ..., n do
2     k ← kleinster Index in {1, ..., j − 1} mit A[j] ≤ A[k]
                                   ▷ A[j] gehört an diese Stelle k
3     x ← A[j]                     ▷ merke A[j]
4     verschiebe A[k, ..., j − 1] nach A[k + 1, ..., j]
5     A[k] ← x
```

In der j-te Iteration auf der Zeile 2 wir suchen der kleinster Index in $\{1,\ldots,j-1\}$ mit $A[j] \leq A[k]$.
Aus der Invariante, wissen wir, dass nach j-1 Iterationen $A[1,\ldots,j-1]$ sortiert ist. Das bedeutet, dass wir Binäre Suche benutzen können um k zu finden => Laufzeit $O(\log j-1)$

Beispiel. *Betrachten wir das Array $A = (5, 3, 7, 1, 4)$. Insertion-Sort führt nun die folgenden Schritte durch:*

$$\begin{aligned} j = 2: &\quad (5,3,7,1,4) \to (3,5,7,1,4) \\ j = 3: &\quad (3,5,7,1,4) \to (3,5,7,1,4) \\ j = 4: &\quad (3,5,7,1,4) \to (1,3,5,7,4) \\ j = 5: &\quad (1,3,5,7,4) \to (1,3,4,5,7) \end{aligned}$$

Wir beobachten, dass Insertion-Sort in diesem Fall vier Durchläufe benötigt.

Laufzeit:

- $O(n \log n)$ Vergleiche
- $O(n^2)$ Vertauschungen

1.5. Mergesort

Wir teilen A auf zwei Teile, wir Sortieren beide Teile unabhängig von einander. Dann mergen wir diese zwei Teilarrays un einen sortierten Array zu Bekommen.

Algorithmus:

MERGESORT$(A[1..n], l, r)$ ▷ *sortiert* $A[l, \ldots, r]$ MERGESORT

```
1 if l < r then
2     m ← ⌊(l + r)/2⌋
3     MERGESORT(A, l, m)          ▷ sortiere linke Hälfte
4     MERGESORT(A, m + 1, r)      ▷ sortiere rechte Hälfte
5     MERGE(A, l, m, r)           ▷ verschmelze beide Hälften
```

Beispiel. *Betrachten wir das Array* $A = (9, 7, 3, 2, 1, 8, 4, 6)$ *und die Indizes* $l = 1$, $m = 4$ *und* $r = 8$. *Die Teilarrays* $A[1, \ldots, 4] = (9, 7, 3, 2)$ *und* $A[5, \ldots, 8] = (1, 8, 4, 6)$ *werden zunächst rekurisv sortiert.*

Mergesort führt also die folgenden Schritte durch:

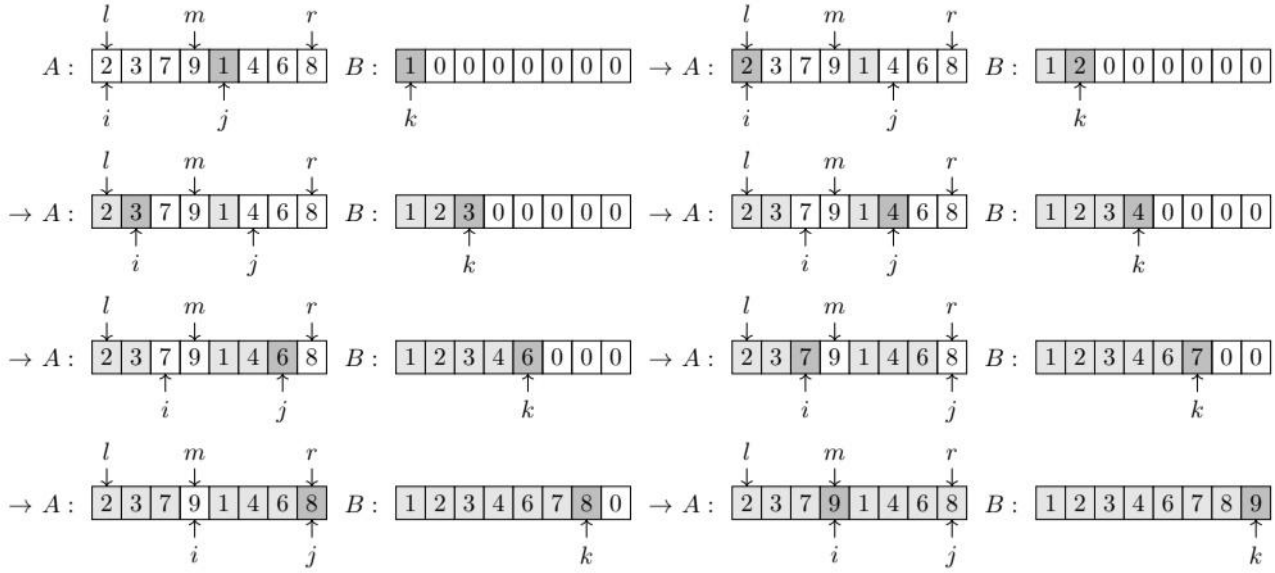
$(9, 7, 3, 2,\ 1, 8, 4, 6)$
$\to (2, 3, 7, 9,\ 1, 4, 6, 8)$ *sortiere beide Hälften*
$\to (1, 2, 3, 4, 6, 7, 8, 9)$ *merge*

MERGE

MERGE$(A[1..n], l, m, r)$

```
1  B ← neues Array mit r − l + 1 Zellen   ▷ so gross wie A[l, ..., r]
2  i ← l                                  ▷ erstes unbenutztes Element in linker Hälfte
3  j ← m + 1                              ▷ erstes unbenutztes Element in rechter Hälfte
4  k ← 1                                  ▷ nächste Position in B
5  while i ≤ m and j ≤ r do               ▷ beide Hälften noch nicht ausgeschöpft
6      if A[i] < A[j] then
7          B[k] ← A[i]
8          i ← i + 1
9          k ← k + 1
10     else
11         B[k] ← A[j]
12         j ← j + 1
13         k ← k + 1
14 übernimm Rest links bzw. rechts        ▷ wenn die andere Hälfte ausgeschöpft ist
15 kopiere B nach A[l, ..., r]
```

Beispiel. *Die folgende Graphik veranschaulicht die Arbeitsweise von Merge auf dem Beispiel von oben.*



Laufzeit: $O(n \log n)$

1.6 Quicksort

Wir wählen ein Pivot-Element A[q] und wir teilen A auf zwei Gruppen:

- Linke Gruppe: Elemente aus A, die kleiner als oder gleich A[q] sind
- Rechte Gruppe: Elemente aus A, die größer als A[q] sind

Algorithmus

QUICKSORT

```
QUICKSORT(A[1..n], l, r)
1 if l < r then
2     k ← Aufteilen(A, l, r)        ▷ Teile A[l..r] in zwei Gruppen auf
3     Quicksort(A, l, k − 1)        ▷ Sortiere linke Gruppe
4     Quicksort(A, k + 1, r)        ▷ Sortiere rechte Gruppe
```

AUFTEILEN

```
AUFTEILEN(A[1..n], l, r)
 1 p ← A[r]                          ▷ Pivotelement
 2 k ← Zahl der Elemente ≤ p in A[l..r]
 3 B ← neues Array mit r − l + 1 Zellen   ▷ so gross wie A[l, ..., r]
 4 B[k] ← p                          ▷ Pivot muss an k-te Stelle
 5 i ← l                             ▷ Anfang des linken Teils von B
 6 j ← k + 1                         ▷ Anfang des rechten Teils von B
 7 for s ← l, l + 1, ..., r
 8     if A[s] ≤ p then
 9         B[i] ← A[s]               ▷ Schreibe A[s] in linke Hälfte
10         i ← i + 1
11     else
12         B[j] ← A[s]               ▷ Schreibe A[s] in rechte Hälfte
13         j ← j + 1
14 kopiere B nach A[l..r]
```

Alternative Implementierung von Aufteilen

AUFTEILEN($A[1..n], l, r$) — AUFTEILEN

```
1  i ← l                          ▷ Index für die linke Gruppe
2  j ← r − 1                      ▷ Index für die rechte Gruppe
3  p ← A[r]                       ▷ Pivotelement
4  repeat
5      while i < r and A[i] ≤ p do
6          i ← i + 1              ▷ Suche nächstes Element für rechte Gruppe
7      while j > l and A[j] > p do
8          j ← j − 1              ▷ Suche nächstes Element für linke Gruppe
9      if i < j then
10         Vertausche A[i] und A[j]   ▷ Vertausche Elemente
11 until i ≥ j                    ▷ Schleife endet, wenn sich i und j „treffen“
12 Vertausche A[i] und A[r]       ▷ Pivotelement an die richtige Stelle tauschen
13 return i                       ▷ Gib Position des Pivotelements zurück
```

Laufzeit: Die Laufzeit hängt von den Auswahl von dem Pivot-Element ab.
Beim schlechten Auswahl wird A so geteilt, dass eine Gruppe viel mehr elemente als die Andere Gruppe hat. In diesem Fall ist die Laufzeit $O(n^2)$.
Beim guten Auswahl wird A in zwei Gruppen geteilt, die ungefähr gleich Anzahl von Elemente haben. Dann ist die Laufzeit $O(n \log n)$.

1. F. Max-Heap

Ein *Max-Heap* ist ein binärer Baum, in dem jeder Knoten einen *Schlüssel*[5] hat und der die folgenden beiden Bedingungen erfüllt:

1. *Vollständiger Binärbaum*: Jeder Knoten hat genau zwei Kinder, ausser eventuell in den beiden untersten Levels des Baums. Das unterste Level ist von links nach rechts aufgefüllt. Somit haben Knoten im untersten Level keine Kinder, und im zweituntersten Level können Knoten mit 0, 1, oder 2 Kindern auftreten.

2. *Heap-Bedingung*: Der Schlüssel jedes Knotens ist grösser oder gleich den Schlüsseln seiner Kinder.

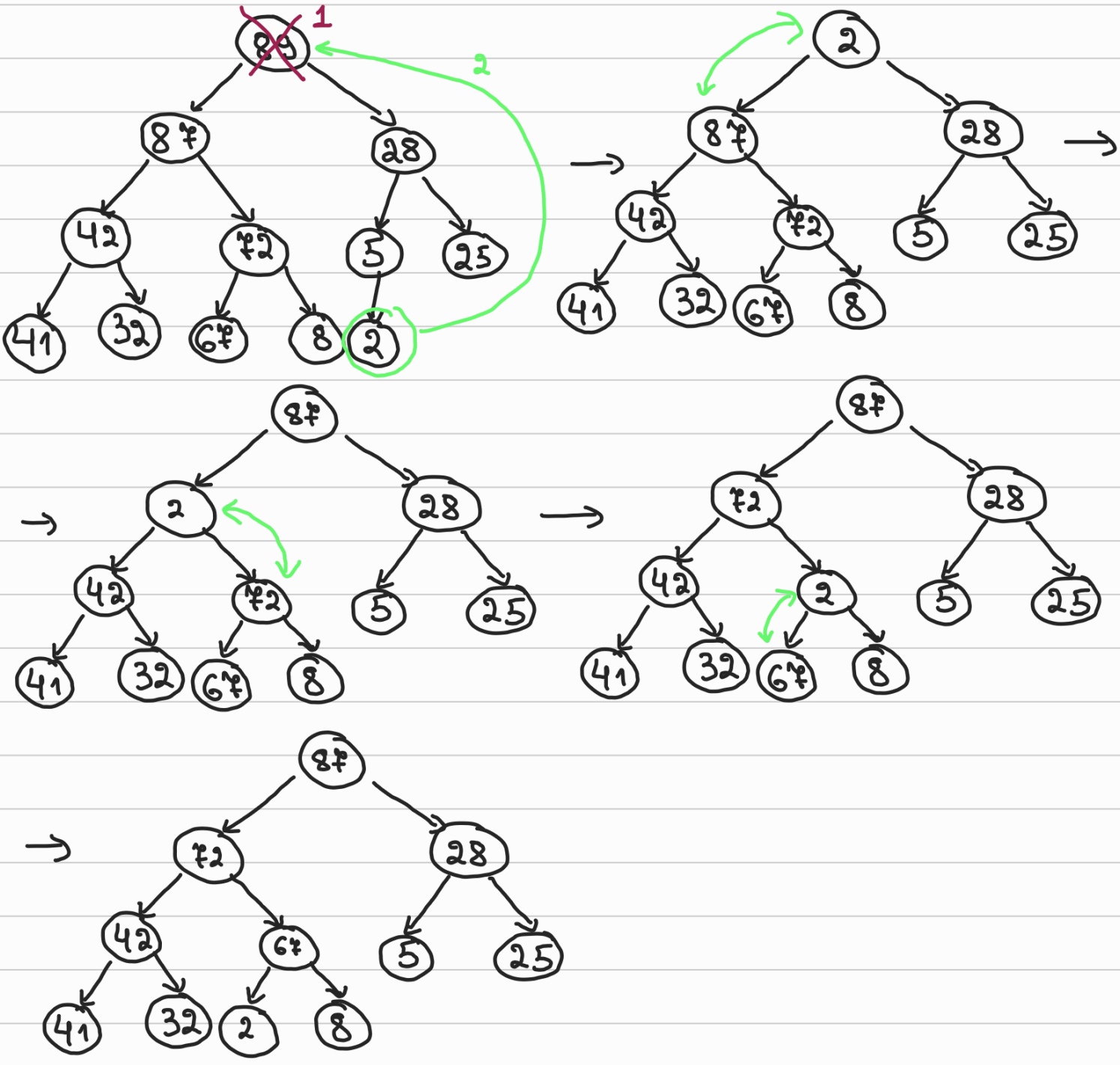
Maximum aus Heap löschen

Die Operation ExtractMax besteht aus zwei Schritten:

1. Entferne die Wurzel des Heaps. Dadurch entsteht eine Lücke an der Wurzelposition, und die Baumstruktur des Heaps wird verletzt.

2. Verschiebe das letzte Blatt des Heaps an die Wurzelposition.[6] Dadurch wird die Baumstruktur wieder hergestellt, allerdings ist die Heap-Bedingung möglicherweise verletzt. Stelle die Heap-Bedingung wieder her, indem du den neuen Wurzelknoten so lange mit dem grösseren seiner beiden Kinder vertauschst ("versickern lässt"), bis die Heap-Bedingung erfüllt ist.

Laufzeit: $O(\log n)$



Daten in Heap umwandeln

Einfügen in Heap (*insert*)

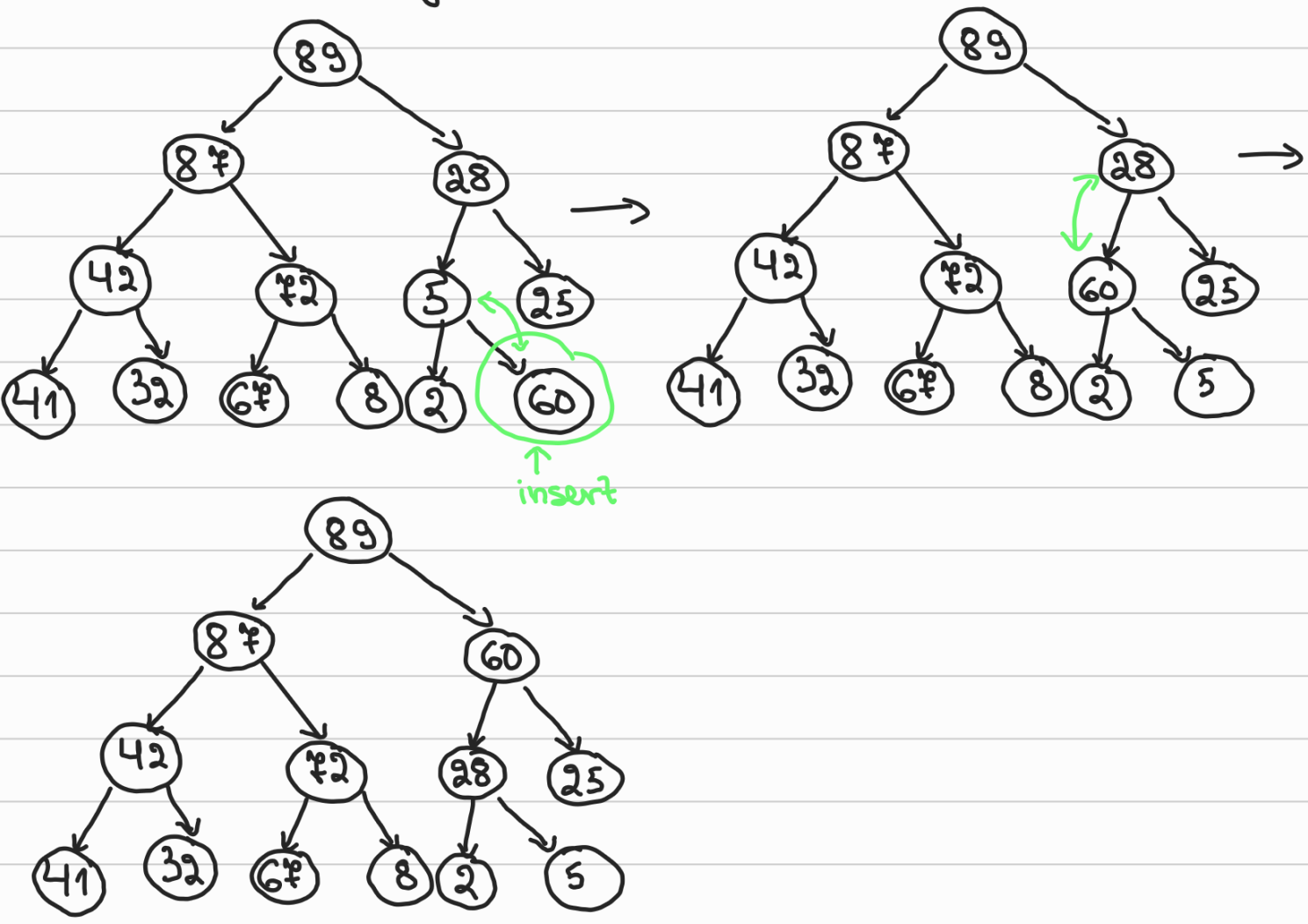
Die Operation insert(H, p) fügt einen neuen Knoten mit Schlüssel p in den Heap H ein. Sie besteht aus zwei Schritten:

1. Füge einen neuen Knoten v an der nächsten freien Stelle im Heap ein. Dadurch wird die Baumstruktur des Heaps erhalten.

2. Stelle die Heap-Bedingung wieder her, indem du den neuen Knoten v so lange mit seinem Elternknoten vertauschst, bis die Heap-Bedingung erfüllt ist.

Analog zum Versickern überlegt man sich, dass Operation 2. die Heap-Bedingung wieder herstellt. Der einzige Knoten, der die Heap-Bedingung möglicherweise verletzt, ist der jeweilige Elternknoten von v. Das Problem verschwindet spätestens, wenn v zur Wurzel des Baumes aufsteigt, da es dann keinen Elternknoten mehr gibt.

Laufzeit: $O(\log n)$



2. Moodle Quiz Besprechung

Aufgabe 1

In a nonempty max-heap, which node is guaranteed to contain a largest key? Select one answer.

a. The rightmost leaf.
b. The root.
c. The leftmost leaf.

Lösung: Bei Max-Heap alle Eltern haben Schüssel grösser als die Schlüsseln von alle von seine Kinder. Das bedeutet, dass das Wurzel besitzt das grösste Schlüssel.

Aufgabe 2

In a singly linked list, what does a node's next pointer store when the node has a successor? Select one answer.

a. The position of the node in sorted order.
b. The memory address of the next node.
c. The key of the previous node.

Lösung: b) The memory address of the next node

Aufgabe 3

Given an upper bound on the list's size, the abstract data type List can be implemented using either an array or a linked list.

True
False

Lösung: True

Aufgabe 4

Binary Search looks for 4 in $A = (1, 3, 5, 7, 9, 11, 13)$. Which sequence of values does it inspect? Select one answer.

- a. $(7, 3, 1)$
- b. $(7, 3, 5)$
- c. $(7, 5, 3)$

Lösung:

Iteration 1: $\ell = 1$, $r = 7$, $m = \lfloor (1+7)/2 \rfloor = 4$

$\Rightarrow$ wir vergleichen 4 mit $A[4] = 7$

$4 < 7$

$\Rightarrow \ell = 1, r = 3$

Iteration 2: $\ell = 1, r = 3$, $m = \lfloor (1+3)/2 \rfloor = 2$

$\Rightarrow$ wir vergleichen 4 mit $A[2] = 3$

$4 > 3$

$\Rightarrow \ell = 3, r = 3$

Iteration 3: $\ell = 3, r = 3$, $m = \lfloor (3+3)/2 \rfloor = 3$

$\Rightarrow$ wir vergleichen 4 mit $A[3] = 5$

$4 < 5$

$\Rightarrow \ell = 3, r = 2$

$\ell < r \Rightarrow$ wir können nicht weiter machen

$\Rightarrow$ B) $(7, 3, 5)$

Aufgabe 5

Quicksort has just partitioned an array of distinct integers around a pivot, as in the lecture. For each statement, decide whether it is guaranteed to hold.

Wahr	Falsch		
[redacted]	[redacted]	The entries to the left of the pivot are already sorted.	✓
		Every entry to the left of the pivot is smaller than every entry to its right.	✓
		The pivot is in its final sorted position.	✓

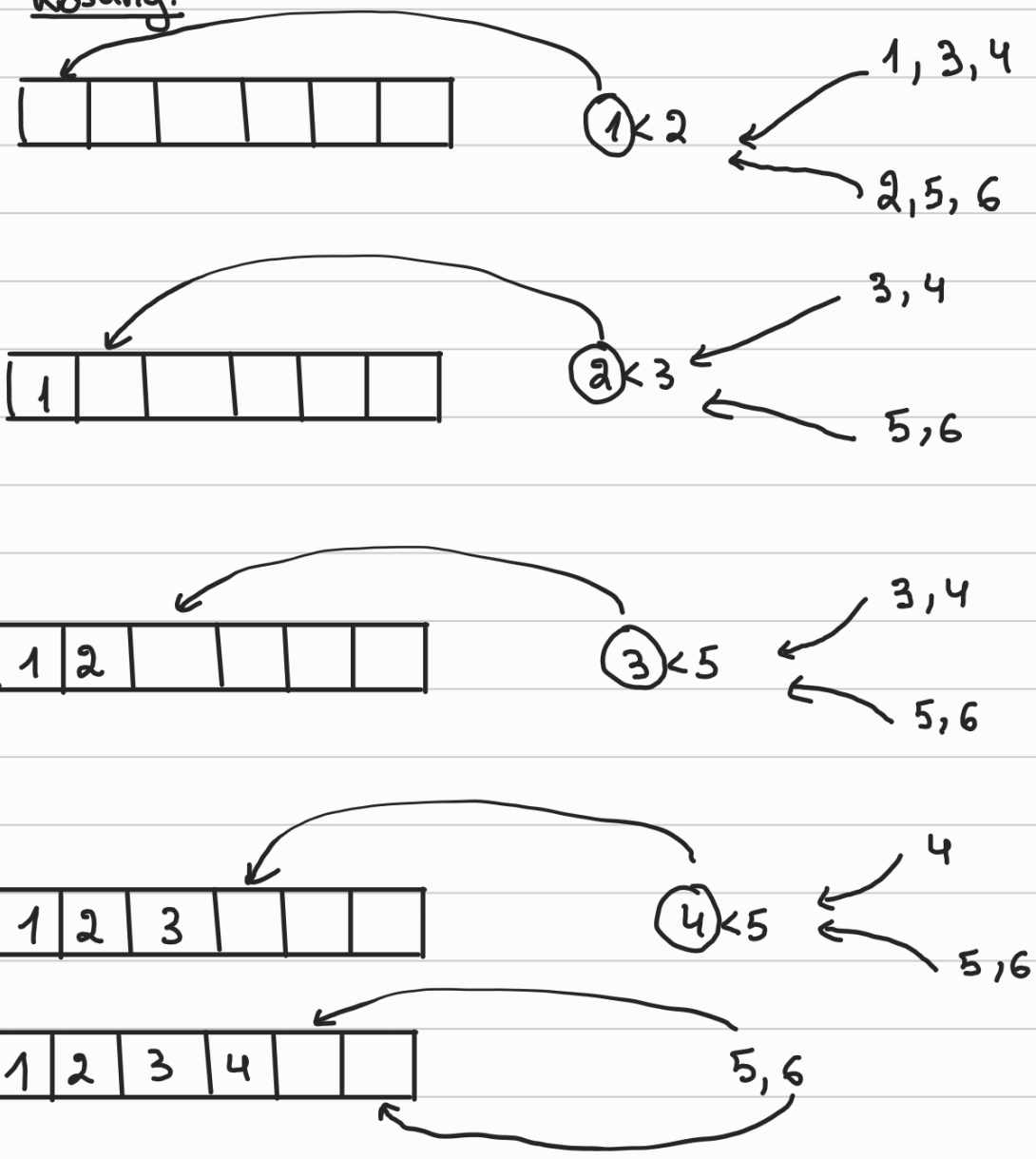
Lösung: 1) Falsch
2) Wahr
3) Wahr

Aufgabe 6

How many comparisons between array entries does the lecture's Merge algorithm perform when merging $(1, 3, 4)$ and $(2, 5, 6)$? Select one answer.

- a. 5
- b. 3
- c. 4

Lösung:



Insgesamt: 4 Vergleichungen. ⇒ c.)

Aufgabe 7

During Insertion Sort, the already sorted prefix must contain the smallest entries of the entire array.

- True
- False

Lösung:

Invariante von Insertion Sort:
Nach j Durchläufen der äusseren Schleife ist das Teilarray A[1,...,j] sortiert (es enthält nich zwangsläufig die j kleinsten Elemente des Arrays)
=> False

Aufgabe 8

How many nonempty contiguous substrings of the bitstring "010" contain exactly one 1?

Answer:

Lösung:

Länge 1: "1"
Länge 2: "01", "10"
Länge 3: "010"
Insgesamt 4

Aufgabe 9

Bubble Sort starts with $A = (2, 4, 1, 3, 5)$ and makes left-to-right passes. After how many complete passes is the array first sorted?

Answer:

Lösung:

Erste pass: 2, 1, 3, 4, 5

Zweite pass: 1, 2, 3, 4, 5 → Sortiert ⇒ Antwort: 2

Aufgabe 10

How many integer multiplications does the iterative-squaring algorithm from Homework 3 use to compute a^8?

Answer: [redacted]

Lösung:

Algorithmus: Power (a, n) Annahme $n = 2^k$

```
if n = 1 then
    return a
x ← Power(a, n/2)
return x * x
```

```
Power(a, 8)
| x ← Power(a, 4)
|     | x ← Power(a, 2)
|     |     | x ← Power(a, 1)
|     |     |     | return a
|     |     | return x * x   (= a * a)
|     | return x * x   (= a² * a²)
| return x * x   (= a⁴ * a⁴)
```

⇒ Insgesamt 3 Integer Multiplikationen

Aufgabe 11

Let $T(1) = 1$ and $T(n) = T(n/2) + n$ for powers of two $n > 1$. Which tight asymptotic bound holds for $T(n)$? Select one answer.

- a. $T(n) = \Theta(n \log n)$.
- b. $T(n) = \Theta(n)$.
- c. $T(n) = \Theta(\log n)$.

Lösung: $T(n) = T(n/2) + n$ mit $n = 2^k$

$$= T\left(\frac{n}{4}\right) + \frac{n}{2} + n$$

$$= T\left(\frac{n}{8}\right) + \frac{n}{4} + \frac{n}{2} + n$$

$$\vdots$$

$$= T\left(\frac{n}{2^i}\right) + \frac{n}{2^{i-1}} + \frac{n}{2^{i-2}} + \dots + \frac{n}{2^0}$$

$$= T\left(\frac{n}{2^k}\right) + \frac{n}{2^{k-1}} + \frac{n}{2^{k-2}} + \dots + \frac{n}{2^0}$$

$$= T(1) + \sum_{i=1}^{k} \frac{n}{2^{k-i}}$$

$$= T(1) + \sum_{i=1}^{k} \frac{2^k}{2^{k-i}}$$

$$= T(1) + \sum_{i=1}^{k} 2^i$$

$$= 1 + \sum_{i=1}^{k} 2^i = 2^0 + \sum_{i=1}^{k} 2^i = \sum_{i=0}^{k} 2^i = 2^{k+1} - 1$$

$$= 2 \cdot 2^k - 1 = 2n - 1 = \Theta(n) \Rightarrow \text{B.}$$

Aufgabe 12

Let n range over integers with $n \geq 5$. For each statement, decide whether it is true or false.

Wahr	Falsch		
[redacted]	[redacted]	$2^n = \Theta(4^n)$.	✓
		$\log_2 n = \Theta(\log_4 n)$.	✓
		$2^{n+1} = \Theta(2^n)$.	✓

Lösung:

1. $\lim_{n\to\infty} \frac{2^n}{4^n} = \lim_{n\to\infty} \left(\frac{1}{2}\right)^n = 0 \Rightarrow 2^n \neq \Theta(4^n)$ und die Aussage ist Falsch

2. $\lim_{n\to\infty} \frac{\log_2 n}{\log_4 n} = \lim_{n\to\infty} \frac{\frac{\log_2 n}{\log_2 4}}{\log_2 n} = \lim_{n\to\infty} \frac{\log_2 4 \cdot \log_2 n}{\log_2 n} = \log_2 4 = 2 \in \mathbb{R}^+$

$\Rightarrow \log_2 n = \Theta(\log_4 n)$ und die Aussage ist Wahr

3. $\lim_{n\to\infty} \frac{2^{n+1}}{2^n} = 2 \in \mathbb{R}^+ \Rightarrow 2^{n+1} = \Theta(2^n)$ und die Aussage ist Wahr.

Aufgabe 13

Let $n = 2^k$ for an integer $k \geq 2$. How many calls to f does the following code make? Select one answer.

$i \leftarrow 1$

while $i \leq n$:

  $j \leftarrow 1$

  while $j \leq i$:

    $f()$

    $j \leftarrow 2j$

  $i \leftarrow 2i$

a. $\frac{k(k+1)}{2}$

b. 2^k

c. $\frac{(k+1)(k+2)}{2}$

d. $2^{k+1} - 1$

Lösung: Erste while-Schleife macht k+1 Iterationen und die zweite while-Schleife macht log i Iterationen

$$\Rightarrow \sum_{i=1}^{k+1} \sum_{j=1}^{i} 1 = \sum_{i=1}^{k+1} i = \frac{(k+1)(k+2)}{2} \Rightarrow c.$$

3. Häufige Fehler aus Exercise Sheet 3

Aufgabe 3.2.

Wenn wir die Laufzeit eines Algorithmus analysieren, reicht es nicht aus anzugeben, wie viele Iterationen eine Schleife durchläuft, wir müssen auch die Laufzeit der einzelnen Iterationen spezifizieren.

c)

```
Algorithmus Spanning(m, k, S)
prefix ← Prefixtable(S[m+1, ..., n-1])
suffix ← Suffixtable(S[1, ..., m])
countSubstrings ← 0
for i = 1 ... k                          (1)
    countSubstrings += prefix[k-i] * suffix[i]   (2)
return countSubstrings
```

(1) In der Aufgabe steht nicht, dass $k < n$. Das bedeutet, dass k gleich n^2 sein kann, wodurch die Laufzeit quadratisch statt linear wird. Wir wissen, dass S n Elemente enthält und somit höhstens n Einsen. Das bedeutet, dass wir, selbst wenn k gräßer als n ist, nicht über n hinaus iterieren müssen.

(2) Zusätzlich müssen wir überprüfen, dass wir nicht out-of-bound gehen.

Bsp. $n = 10$, $k = 5$, $m = 2$

=> prefix [0, ..., 7] -> hat 8 Elemente

und suffix [0, ..., 3] -> hat 4 Elemente

$k = 5$ => i kann 4 und 5 werden, aber

suffix hat Indexen Bis 3 ⇒ wir gehen out of bounds

Algorithmus Spanning(m, k, S)

```
prefix ← Prefixtable(S[m+1, ..., n-1])
suffix ← Suffixtable(S[1, ..., m])
countSubstrings ← 0
for i = 1 ... min(n, k)
    if i ≤ m+1 && k-i ≤ n-m-1
        countSubstrings += prefix[k-i] * suffix[i]
return countSubstrings
```

Aufgabe 3.3.b)

Algorithm 2

(b)
```
i ← 1
while i ≤ n do
    j ← 1
    while j ≤ i^3 do
        f()
        j ← j + 1
    i ← i + 1
```

Hint: *You may use* $\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4}$.

Viele haben den Hinweis direkt andevendet, aber wir müssen erklären, woher die Sume $\sum_{i=1}^{n} i^3$ stammt.

✗ $\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4} = \Theta(n^4)$

✓ $\sum_{i=1}^{n} \sum_{j=1}^{i^3} 1 = \sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4} = \Theta(n^4)$

4. In-Class Exercises

Übung 1.

Sei $S_n = \sum_{i=1}^{n} i^k$. Berechnen Sie $r \in \mathbb{N}$ so dass $S_n = \Theta(n^r)$ für alle $n \in \mathbb{N}$

Exercise 1

In the last exercise class we mentioned that Bubble-Sort **doesn't** know that the input array is sorted until the algorithm terminates. Therefore, the runtime of this algorithm depends only on the input length and not on the order of the elements in the array. For example, Bubble-Sort has the same runtime for the arrays 1, 2, 3, 4, 5 and 5, 3, 1, 4, 2, even though the array 1, 2, 3, 4, 5 is already sorted. Can you suggest an alteration to the algorithm Bubble-Sort that recognizes the point when the array is already sorted and the algorithm can terminate avoiding further unnecessary operations? The altered algorithm should have worst case runtime $O(n^2)$. Is it possible to find a similar alteration for Selection-Sort?

Exercise 2

What would be the runtime of Quicksort given the input array: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1? Is it O(n log n) or something else? Argue why.

Exercise 3

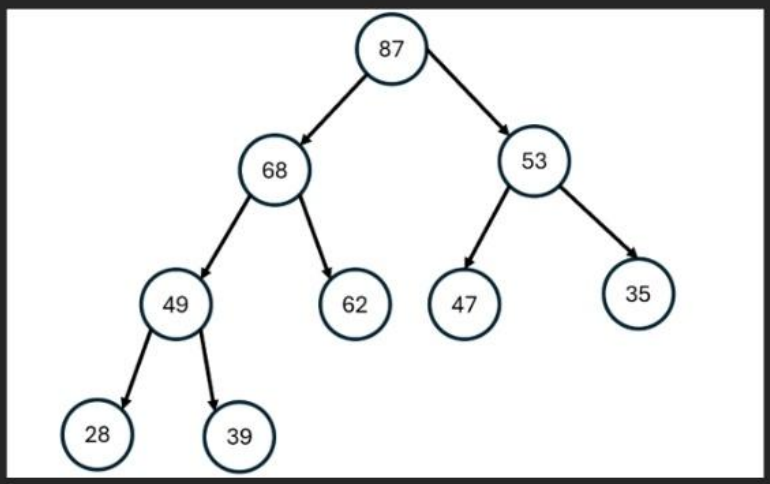
You are given a mystery comparison sorting algorithm. The array 10, 8, 3, 6, 7, 9, 5, 4, 2, 1 is given as an input of the mystery algorithm. After 2 steps of the mystery algorithm the array looks like:

a) 3, 6, 7, 8, 5, 4, 2, 1, 9, 10
b) 8, 10, 3, 6, 7, 9, 5, 4, 2, 1
c) 1, 8, 3, 6, 7, 2, 5, 4, 9, 10

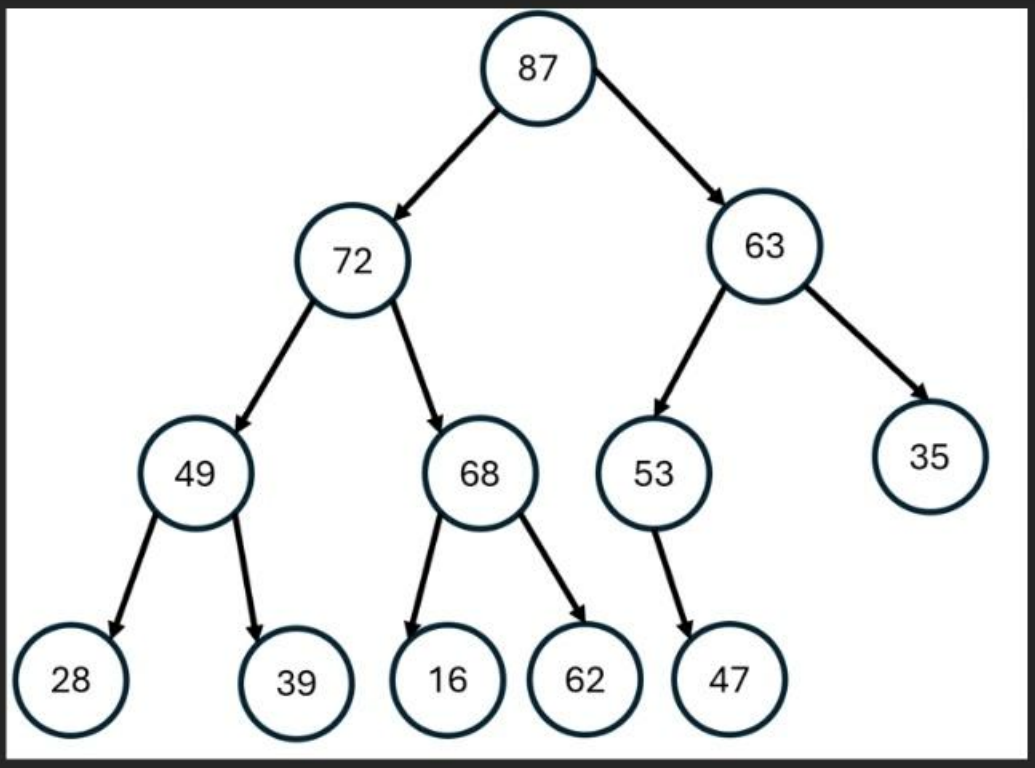
Can you guess the mystery algorithm?

Exercise 5

You are given the following Max-Heap.



Now we insert the keys 16, 72, 63 in this order. What does the Max-Heap look like?



Now we extract the max key 4 times. What does the Max-Heap look like?

